

CSCE 636 DEEP LEARNING · MODULE 08 · LECTURE NOTES

Training at Scale

When the model no longer fits.

MODULE OUTCOMES

- Explain data, tensor, and pipeline parallelism.
- Explain mixed precision and why loss scaling is needed.
- Apply gradient accumulation and checkpointing.
- Reason about the memory budget of a training run.
- Diagnose whether you are compute- or memory-bound.

1 The memory budget

For P parameters, mixed precision, Adam:

parameters	(fp16)	2P bytes
gradients	(fp16)	2P
Adam m	(fp32)	4P
Adam v	(fp32)	4P
fp32 master copy		4P

		16P bytes, BEFORE any activations

A 7-BILLION-PARAMETER MODEL: ~112 GB of state alone. That does not fit on any single accelerator.

ACTIVATIONS are retained for the backward pass and scale with batch $\times$ sequence length $\times$ depth. At long context they frequently EXCEED the parameter memory.

LEVERS, in the order to reach for them:

gradient accumulation	smaller batch, same effective batch
activation checkpointing	recompute rather than store
mixed precision	halve the activation memory
optimiser sharding (ZeRO)	split m, v and master across devices
then actual model parallelism	

The optimiser state is twelve of those sixteen bytes, which is three-quarters of the budget and is reliably the surprise — people size their hardware against the parameter count and are short by a factor of eight. It is also why ZeRO and FSDP exist: sharding the optimiser state across devices attacks the largest term first.

GRADIENT ACCUMULATION AND CHECKPOINTING COME FIRST

Gradient accumulation: run several small micro-batches, accumulate their gradients without stepping, then take one optimiser step. **The result is mathematically identical to a single large batch** (modulo batch normalisation, which is batch-dependent — Module 04 §3), at a fraction of the activation memory.

Activation checkpointing: store the activations of only some layers and recompute the rest during the backward pass. **Roughly $\sqrt{n}$ memory for about 30% additional compute** — an excellent trade whenever memory is the binding constraint, and a bad one when it is not.

Both are a few lines of code and neither changes the mathematics, so they should be fully exhausted before anything distributed is attempted. Distribution brings communication, debugging across processes, and a class of bugs that single-device training does not have.

And they compose. Together they frequently turn a model that will not fit into one that trains comfortably on the hardware you already have — **which is a better outcome than adding a second machine**, and is the same 'do you actually need to distribute?' question CSCE 678 Module 01 §4 asked.

2 Parallelism

Kind	What it splits	Communication cost
Data parallel	The batch, across devices. Every device holds a complete model copy.	An all-reduce of the gradients every step — see below. Simple, and the memory per device is unchanged.
ZeRO / FSDP	The optimiser state, the gradients, and eventually the parameters themselves, across devices.	More communication, dramatically less memory. Parameters are gathered just before use and released after, which trades bandwidth for capacity.
Tensor parallel	Individual weight matrices, within a layer — each device computes part of one matrix multiply.	Communication at every layer, so it requires a very fast interconnect (NVLink within a node) and does not work well across ordinary networks.
Pipeline parallel	Consecutive layers, across devices.	Only at stage boundaries, which is cheap — but it introduces a pipeline bubble, since the last stage idles until the first micro-batch reaches it. Micro-batching reduces but does not remove it.
Expert parallel	The experts of a mixture-of-experts layer.	Routed and sparse — only the selected experts activate, so parameters scale without proportional compute.

Use data parallelism first, then ZeRO, then pipeline, then tensor — in increasing order of communication demand and implementation pain. Large training runs combine all of them (3D parallelism), which is where the engineering genuinely becomes hard.

DATA PARALLELISM IS THE ALL-REDUCE FROM CSCE 735

Each device holds a complete copy of the model and a slice of the batch, computes gradients on its own slice, and then the gradients are averaged across all devices before the optimiser step.

That average is exactly an all-reduce (CSCE 735 Module 10 §2), and **ring all-reduce is what NCCL and the other collective libraries implement** — bandwidth-optimal, and it is the same algorithm the HPC community developed decades earlier.

So the cost is the gradient size divided by the interconnect bandwidth, every single step, which is why NVLink within a node and InfiniBand between nodes matter so much for training throughput — and why gradient compression is an active area.

And it can be overlapped with computation: start reducing the last layer's gradients as soon as they exist, while the earlier layers are still computing theirs. **This is exactly CSCE 735 Module 10 §3's computation–communication overlap**, applied to a problem that course did not anticipate — and like there, **you must measure whether the overlap actually happened** rather than assuming the library arranged it.

3 Mixed precision

WHY LOSS SCALING IS NECESSARY

fp16 has a narrow exponent range. Its smallest normal positive value is about 6×10^{-5} , and subnormals extend only to about 6×10^{-8} .

Gradients are frequently smaller than that, particularly in early layers of deep networks — so they **flush to zero and the learning signal silently disappears**. The model trains, slowly and badly, with no error reported.

So multiply the loss by a large constant before the backward pass. By the chain rule, every gradient is scaled by the same factor, lifting them into the representable range; divide the gradients by that factor before the optimiser step and the mathematics is unchanged. **Dynamic loss scaling adjusts the constant automatically** — raise it when no overflow has occurred for a while, halve it and skip the step when one does.

bf16 avoids the problem entirely. It has the same eight-bit exponent as fp32 — hence the same range — with fewer mantissa bits. **No loss scaling is required at all**, which removes a whole class of bugs, **and it is why bf16 is preferred wherever the hardware supports it.**

Format	sign / exponent / mantissa	Character
fp32	1 / 8 / 23	The baseline. Rarely needed throughout a training run; usually retained only for the master weights and the optimiser state.
fp16	1 / 5 / 10	Narrow range, so it requires loss scaling (above). More mantissa precision than bf16, which is almost never the binding concern.
bf16	1 / 8 / 7	fp32's exponent range with much less precision. The preferred training format — see below for why that trade is the right one.
fp8	1 / 4 / 3, or 1 / 5 / 2	Inference, and increasingly training on hardware that supports it, with per-tensor scaling factors.
int8 and below	Integer plus a scale factor.	Inference only — Module 13 §2.

bf16 trades mantissa bits for exponent bits, and that is the right trade for gradients: their *magnitude* varies over many orders of magnitude between layers and over training, while their *precision* matters very little — the update is going to be averaged over a batch and scaled by a learning rate anyway. **Knowing why the format was designed that way makes the choice obvious rather than received.**

4 Diagnosing a slow training run

- **Measure the achieved FLOPs against the device's peak.** Below roughly 30% of peak means you are not compute-bound, and optimising the arithmetic will achieve nothing — which is CSCE 735 Module 07's roofline argument applied directly.
- **Measure memory bandwidth utilisation.** High bandwidth with low FLOPs means memory-bound, and the fix is fusion, better layout, or larger batches — not a faster kernel.
- **Measure the fraction of step time spent in communication.** If it is large, overlap it with computation (§2), reduce the gradient size, or reconsider the parallelism strategy.
- **Check the data loader. A GPU sitting idle waiting for the next batch is extremely common and is completely invisible unless you look for it** — the training loop reports normal-looking step times and the accelerator is at 20% utilisation. **It is the single most common bottleneck in small-scale training and almost nobody checks it first.** More worker processes, prefetching, and doing augmentation on the GPU all help.
- **And profile before optimising** — CSCE 735 Module 07, entirely unchanged. The profiler names the limiting resource; guessing does not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Rajbhandari et al. — ZeRO: Memory Optimizations Toward Training Trillion Parameter Models (free)	The §1 memory accounting and the §2 sharding strategy , with the three stages laid out clearly.
Mickevicius et al. — Mixed Precision Training (free)	The §3 loss-scaling argument , including the gradient histograms that show the flush-to-zero problem.
Hugging Face — Performance and Scalability documentation (free)	The practical lever ordering of §1 , with measured effects for each technique.
Chen et al. — Training Deep Nets with Sublinear Memory Cost (free)	Activation checkpointing and the $\sqrt{n}$ result.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute the memory budget** for a model you want to train, using the 16P formula, and compare against your hardware.
- 2 **Measure actual GPU memory** during a training step and break it down by category.

- 3 Implement gradient accumulation and verify the result matches a large batch exactly.
- 4 **Enable activation checkpointing** and measure the memory saving and the compute cost.
- 5 Train in fp32, fp16 with loss scaling, and bf16. Compare final loss and wall-clock time.
- 6 **Disable loss scaling in fp16** and plot the fraction of gradients that flush to zero.
- 7 Run data-parallel training on two devices and measure the all-reduce time per step.
- 8 **Verify whether gradient reduction overlaps with the backward pass**, as CSCE 735 Module 10 required.
- 9 **Profile your data loader** and report the fraction of step time the GPU spends idle.
- 10 Place your training step on a roofline and state what limits it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the memory cost per parameter for mixed-precision Adam, broken down.
- 2 What do gradient accumulation and checkpointing each cost and save?
- 3 Compare five kinds of parallelism and give the order to use them.
- 4 Why is data parallelism an all-reduce, and what can be overlapped?
- 5 Why does fp16 need loss scaling, and how does dynamic scaling work?
- 6 Why does bf16 avoid the problem, and why is its trade the right one?
- 7 Give five things to measure when training is slow.
- 8 What is the most common small-scale bottleneck?